

# 전송 계층(Transport Layer) 연습문제 학습서

교재 원본 분석 기반 정답 복원 및 상세 해설 집합본

## I. 객관식 문항 정밀 해설

### [문제 1] 전송 계층과 데이터 링크 계층의 비교 (원본 11번)

- ① 오류 제어, 흐름 제어, 데이터 순서화 등의 기능 면에서 두 프로토콜의 특징은 유사하다.
- ② 데이터 링크 계층은 물리적인 전송 선로로 직접 연결된 호스트 사이의 데이터 전송을 담당한다.
- ③ 전송 계층은 네트워크 끝단에 있는 통신 주체 사이의 논리적 선로를 통해 데이터를 주고받는다.
- ④ 데이터 링크 계층은 물리적 선로를 통해 데이터를 전달하기 때문에 네트워크 계층의 중개 기능이 중요하다.
- ⑤ 전송 계층은 중간에 위치한 논리적 네트워크의 중개 기능을 사용해 전송 기능을 수행한다.

정답: ④

해설:

라우터나 스위치 등의 장비를 거치며 패킷을 최적의 경로로 목적지까지 안내하는 '중개 기능(라우팅)'은 **네트워크 계층(Network Layer)**의 핵심 역할입니다. 데이터 링크 계층은 중개 기능 없이 오직 '물리적으로 직접 연결된 인접 노드 간'의 신뢰성 있는 전송만을 관리하므로 ④번 문장은 완전히 잘못되었습니다.

### [문제 2] 전송 계층의 오동작 및 오류 발생 원인 (원본 12번)

- ① 양 끝단의 호스트에서 실행되는 프로세스 사이의 전송 속도는 혼잡 제어 기능이 조절한다.
- ② 전송 오류가 발생하여 데이터의 내용이 깨지거나 분실되면 데이터 재전송에 의한 오류 제어 기능에 의하여 복구 절차가 진행된다.
- ③ 전송 계층에서 발생하는 오류는 논리적으로 구축된 각 계층의 소프트웨어가 동작하는 과정에서 데이터를 분실하는 경우보다는 선로 오류에 기인한 경우가 대부분이다.
- ④ 상위 계층에서 전송을 요구한 데이터 크기가 전송 계층에서 처리할 수 있는 데이터 크기보다 크면 데이터를 쪼개서 전송해야 한다.
- ⑤ 전송 서비스 프리미티브는 전송 계층 사용자가 전송 계층 서비스를 사용하기 위한 인터페이스이다.

정답: ③

해설:

선로 노이즈나 신호 감쇠 등으로 인한 물리적인 선로 오류는 하위 계층(데이터 링크 계층 등)에서 대부분 검출되고 필터링됩니다. **전송 계층 수준에서 발생하는 주요 오류(패킷 누락, 분실 등)는 대개 라우터의 버퍼 용량 초과로 패킷을 폐기하는 혼잡(Congestion) 현상이나 소프트웨어 버퍼 오버플로우 등 상위의 논리적 처리 과정에서 발생합니다.** 따라서 선로 오류가 대부분이라는 ③번은 오답입니다.

### [문제 3] 주소 표현 및 포트 번호의 특징 (원본 17번)

- ① 포트 번호는 TCP와 UDP가 상위 계층에 제공하는 주소 표현 방식이다.
- ② 인터넷 환경에서 많이 사용하는 네트워크 응용 서비스의 서버 프로세스에 할당된 포트 번호는 전 세계 모든 서버가 동일한 포트 번호를 사용하도록 권고되고 있다.
- ③ 전자우편(SMTP)의 Well-known 포트 번호는 25번이고, HTTP는 80번이다.
- ④ 네트워크 서비스를 제공하는 포트 번호는 파일 시스템에 보관되므로 일반 사용자가 포트 번호를 직접 지정하는 경우는 없다.
- ⑤ TCP와 UDP는 별도의 포트 주소 공간을 관리하므로 동일한 포트 번호를 사용할 수 있다.

정답: ①, ②, ③, ⑤

해설:

④번의 경우, 일반 사용자도 웹 브라우저 주소창에 `http://localhost:8080`과 같이 포트 번호를 명시하여 접근할 수 있고, 개발자 역시 소켓 바인딩 시 포트 번호를 직접 지정하므로 '직접 지정하는 경우가 없다'는 설명은 명백한 오류입니다. TCP와 UDP는 전송 계층 프로토콜 자체가 완전히 분리되어 있어 동일 번호의 포트 공간을 각각 독립적으로 보유할 수 있습니다.

### [문제 4] TCP 프로토콜의 전송 메커니즘 (원본 14번)

- ① TCP는 연결형 서비스를 지원하고 UDP는 비연결형 서비스를 제공한다.
- ② TCP는 데이터를 세그먼트(Segment)라고 불리는 블록 단위로 분할해 전송한다.
- ③ TCP는 세그먼트를 하나의 단위로 간주하여 순서 번호를 관리한다. 따라서 10개의 세그먼트를 전송하면 순서 번호도 10이 증가해야 한다.
- ④ TCP의 상위에서는 파일 전송, 파일 공유, 전자 메일 등의 응용 프로그램들이 실행된다.
- ⑤ TCP는 전이중 방식의 양방향 가상 회선을 제공하며, 신뢰성 있는 데이터 전송을 보장한다.

정답: ①, ②, ④, ⑤

해설:

데이터 링크 계층 등에서는 프레임의 개수 단위로 순서 번호를 증가시키지만, TCP의 순서 번호(Sequence Number)는 세그먼트의 개수가 아니라 '전송하는 데이터의 바이트(Byte) 수'만큼 누적되어 증가합니다. 가령 1,000바이트 크기의 세그먼트를 전송하면 다음 세그먼트의 순서 번호는 1,000이 증가하게 됩니다. 따라서 ③번은 오답입니다.

### [문제 5] TCP 헤더 내부 필드의 역할 구조 (원본 15번)

- ① Options 필드와 Padding 필드는 생략 가능하므로 헤더의 최소 크기는 20바이트이다.
- ② Source Port와 Destination Port 필드는 TCP로 연결되는 가상 회선 양단의 송수신 프로세스에 할당된 네트워크 포트 주소이며, TCP와 UDP가 통합된 주소 공간을 갖기 때문에 같은 번호를 사용해선 안 된다.
- ③ Sequence Number 필드는 송신자가 지정하는 순서 번호이며, 전송되는 바이트 수를 기준으로 증가한다.
- ④ Acknowledgement Number 필드는 수신 프로세스가 제대로 수신한 바이트의 수를 응답하기 위해 사용한다. ACK 플래그 비트가 지정된 경우에만 유효하고, 올바른 수신한 데이터의 마지막 순서 번호를 표기한다.
- ⑤ Window 필드는 수신 윈도우의 버퍼 크기를 지정하는 데 사용되며, 수신자가 데이터를 수신할 수 없으면 이 필드 값을 0으로 지정한다.

정답: ①, ③, ⑤

해설:

- ②번 문항은 앞서 설명했듯 주소 공간이 결합되어 있지 않고 독립적이므로 틀린 설명입니다.
- ④번의 경우, 승인 번호(Acknowledgement Number)는 수신 완료한 마지막 바이트 번호를 적는 것이 아니라, "송신자측으로부터 해당 번호 직전까지 정상적으로 수집 완료하였으니, 다음 차례에는 **마지막 수신 바이트 번호 + 1**번 바이트를 보내라"는 의미로 '다음에 수신하기를 기대하는 누적 바이트 번호'를 기록하기 때문에 틀린 보기가 됩니다.

### [문제 6] TCP 신뢰성 보장 및 흐름 제어 (원본 18번)

- ① 데이터를 전송하기 전에 반드시 3단계 설정 방식을 사용하여 연결을 설정해야 한다.
- ② 연결을 설정하고자 하는 프로세스는 TCP 헤더의 SYN 플래그를 지정한 세그먼트를 전송함으로써 연결 설정을 요구한다.
- ③ TCP는 데이터 전송 과정에서 흐름 제어를 지원하기 위하여 슬라이딩 윈도우 프로토콜을 사용한다.
- ④ 데이터 전송 과정에서 프레임 변형 오류가 발생하면 부정 응답 기능을 사용하여 오류를 복구한다.
- ⑤ 연결 해제는 양쪽 프로세스의 동의하에 진행되기 때문에 연결 해제 세그먼트를 받은 프로세스가 FIN 플래그로 응답할 때까지 연결은 계속 유지된다.

정답: ①, ②, ③, ⑤

해설:

TCP는 기본적으로 부정 응답(NAK) 메커니즘을 지원하지 않는 프로토콜입니다. 오직 데이터를 정상적으로 수신했을 때만 긍정 응답(ACK)을 발행하며, 패킷 손실이나 변형이 일어났을 때는 송신 측에서 설정된 타이머 기반의 타임아웃(Timeout)을 통해 패킷을 재전송하거나 3개의 중복 ACK(3 Duplicate ACKs)를 감지해 빠른 재전송을 가동합니다.

## II. 완성형 빈칸 채우기 정답 문장

- 상위 계층에서 요청한 전송 데이터가 전송 계층의 처리 유닛 규격을 상회할 때 진행되는 쪼개기 메커니즘을 **분할(Segmentation)**이라 칭하며, 수신단에서 이를 순서대로 정렬 결합해 원상태로 돌리는 절차를 **재조립(Reassembly)**이라 일컫는다.
- 통신을 개시하기에 앞서 신뢰 공간을 형성하기 위해 수행하는 클라이언트의 연결 요청(SYN), 서버의 수락 응답(SYN+ACK), 클라이언트의 최종 승인(ACK)으로 전개되는 일련의 3단 연결 수립 메커니즘을 **3단계 연결 설정(3-way handshaking)**이라고 한다.
- **TCP** 프로토콜은 비신뢰성인 IP 프로토콜 레이어 위에서 종단 간 연결형 인프라 및 전이중 가상 회선 통신로를 상위 계층에 제공함으로써 데이터의 절대적 신뢰 수송을 달성한다.
- 전송 계층에서 구동하는 애플리케이션 식별자 주소는 **포트(Port)** 번호라 부르며, 전 세계 공용으로 규격화된 표준 포트 영역은 **Well-known 포트**이다. UNIX 시스템 계열에서는 개발자가 제어하기 쉽도록 이를 **소켓(Socket)**이라는 소프트웨어 추상화 객체 인터페이스로 구현한다.
- 가상 연결 수립 단계에서 클라이언트는 헤더 내 **SYN** 제어 플래그 비트를 활성화해 발송하고, 이를 정상 수용하려는 서버는 수락 의미의 **SYN** 비트와 수신 확인의 **ACK** 비트를 동시 활성화하여 응답해야 성립한다.

## III. 서술형 문항 답안 및 분석 가이드

### [질문 24] TCP의 헤더 구조 모델 및 주요 핵심 필드의 기능 분석

TCP 헤더는 기본적으로 선택 옵션을 제외하고 **20 byte**의 고정 최소 길이를 보장합니다. 주요 핵심 필드는 다음과 같습니다.

- **송신 및 수신 포트 주소 (각 16비트):** 출발지와 목적지 호스트 내부에서 구동되는 프로세스를 개별 식별하기 위한 종단 인터페이스 주소입니다.
- **순서 번호 (Sequence Number, 32비트):** 데이터 스트림 내의 각 바이트에 부여되는 고유 일련번호로, 유실 여부 확인 및 수신측에서의 재조립 정렬 기준이 됩니다.
- **승인 번호 (Acknowledgement Number, 32비트):** 상대방으로부터 수신하기 바라는 다음 바이트 번호를 명시하는 필드로, 누적형 확인 응답 방식을 취합니다.
- **제어 플래그 비트 (6비트):** SYN, ACK, FIN, RST 등 연결 상태를 제어하고 동기화하기 위한 제어 시그널 플래그입니다.
- **윈도우 크기 (Window Size, 16비트):** 현재 수신 측에서 수용할 수 있는 남은 여유 버퍼 용량을 송신측에 동적으로 통보하여 오버플로우를 막는 흐름 제어용 필드입니다.

### [질문 27] 정상적인 TCP 데이터 전송 과정과 오류 상황에서의 복구 프로세스 비교

- **정상 전송 시나리오:** 송신자가 데이터 세그먼트(**Seq = 101**, 데이터 길이 **100 byte**)를 내보내면, 수신단은 변형 없이 완벽히 도착함을 인지한 뒤 다음으로 기대하는 위치 번호(**Ack = 201**)와 **ACK** 플래그를 할당한 세그먼트를 회신함으로써 성공적으로 완료됩니다.

- **오류 발생 시나리오 (패킷 유실):** 전송 과정 중 중간 노드(라우터)의 폭주 등으로 세그먼트가 증발되거나 변형되면, 수신자는 어떠한 ACK도 송신자에게 돌려주지 않습니다. 송신자는 세그먼트 발송과 동시에 가동한 **재전송 타이머(RTO)**가 만료될 때까지 응답을 받지 못하면 연결 경로 상에 사고가 발생했음을 인지하고 해당 데이터를 버퍼에서 꺼내 **재전송(Retransmission)** 메커니즘을 구동합니다.

#### [질문 28] 가상 회선 연결 해제 시의 4단계 절차 (4-way handshaking) 논리 메커니즘

TCP는 양방향 통신 선로를 각각 독립적으로 안전하게 차단하기 위해 점진적 해제 구조를 적용합니다.

1. **1단계 (A → B):** 송신을 마친 호스트 A가 연결을 끊기 위해 **FIN** 플래그 세그먼트를 B에 전달하고 **ext{FIN\_WAIT\_1}** 상태로 전이합니다.
2. **2단계 (B → A):** FIN을 받은 B는 확인 응답인 **ACK**를 돌려주고 자신은 **ext{CLOSE\_WAIT}**로 바뀝니다. ACK를 받은 A는 **ext{FIN\_WAIT\_2}**가 되며, 이 시점부터 A에서 B로 가는 선로는 차단되나 B에서 A로의 잔여 전송은 허용되는 **Half-Close** 상태가 됩니다.
3. **3단계 (B → A):** B 역시 모든 잔여 업무 처리를 마친 후, 연결을 완전히 정리하기 위해 A에게 **FIN** 세그먼트를 발송하고 **ext{LAST\_ACK}** 상태에서 대기합니다.
4. **4단계 (A → B):** B의 FIN을 수신한 A는 마지막 수락의 의미로 **ACK**를 회신한 뒤 **ext{TIME\_WAIT}** 상태로 들어갑니다. 혹시 모를 네트워크 지연 패킷의 완전 소멸 및 마지막 ACK 유실 복구를 위해 일정 시간 대기 후 세션을 완벽히 파괴( **ext{CLOSED}**)합니다.

#### [질문 29] TCP/IP 아키텍처 환경 하에서의 혼잡 제어(Congestion Control) 구동 원리

혼잡 제어는 수신자의 상황만을 보는 흐름 제어와 달리, **인터넷 내부 경로상에 위치한 공유 라우터들의 폭주 및 회선 과부하 상태를 추정하여 송신율을 동적으로 조절**하는 거시적 제어 기법입니다. 송신자는 수신자가 허용한 윈도우와 별개로 내부 **혼잡 윈도우(cwnd)** 크기를 관리하여 제어합니다.

- **느린 시작 (Slow Start):** 초기 네트워크 상태 파악을 위해 cwnd를 1로 낮게 출발하지만, ACK가 정상 복귀할 때마다 창 크기를 지수 함수 형태(1, 2, 4, 8...)로 급격히 키워 임계치(ssthresh) 직전까지 도달시킵니다.
- **혼잡 회피 (Congestion Avoidance):** 임계치를 넘어선 이후부터는 혼잡을 예방하기 위해 cwnd를 매 왕복 시간(RTT)마다 선형적으로 완만하게 1씩 확장시키며 정밀하게 조율합니다.
- **혼잡 감지 및 조치:** 통신 중 패킷 타임아웃(Timeout) 현상이 발발하면 네트워크 붕괴 위험 상태로 판단하여 임계치(ssthresh)를 현재 전송량의 **50%**로 급격히 내린 후, cwnd를 다시 최소치인 1로 고꾸라뜨려 트래픽 유입량을 원천 차단한 뒤 다시 조절 단계를 밟아 나갑니다.